

HOMEWORK 3

PPO, TD3 and SAC

Dr. Mustafa Erdem
erdem@tau.edu.tr

Dr. Onur Akgün
akgun@tau.edu.tr

CCNY-11200 Topics in Algorithms

1 Overview

The purpose of this assignment is to implement, train and evaluate reinforcement learning algorithms on navigation tasks on different levels, each represented by different state and action spaces. The reinforcement learning tasks defined in this assignment require achieving policy implementations of three primary approaches depending on the abstraction level of the navigation task:

- In this assignment, the TD3 algorithm will replace the low-level PID controller for the longitudinal and steering control of the vehicle: i.e. it will take the steering angle command coming from the Pure-Pursuit algorithm as input and calculate the torque reference for the steering motor. The reference calculations will also take into consideration sensor based environmental observations affecting the demanded torque in both cases.
- The PPO algorithm will be used for high-level planning and for the generation of the above-mentioned velocity reference and steering reference (replacing PP). These references will then be fed to the PID controller for low-level control.
- The SAC algorithm will be used for end-to-end control; i.e. without any explicit planning algorithm, by simply learning torque references for longitudinal and steering control, using the environmental observations.

In this assignment, you will implement this TD3, PPO and SAC algorithms that frequently appear in the field. For each algorithm, a skeleton code is provided to you. You need to complete the missing parts and then train the learning agents. During training some performance metrics are provided as output to inspect how the learning process is going. For further inspection you can define and utilize other metrics such as policy loss, value loss etc.

After the trainings are finalized, you can evaluate the trained agents on the environment using the saved models.

2 Codebase Structure

You must not modify the files listed under "Provided / Do Not Edit". You must implement the skeleton files listed under "Your Files".

2.1 Provided / Do Not Edit

HW3/

```
__init__.py
algorithms/
  __init__.py
  base.py
  buffers.py
controllers/
  __init__.py
  pid.py
  pure_pursuit.py
dynamics/
  __init__.py
  tire_model.py
  vehicle_dynamics.py
  vehicle_params.py
logs/
models/
config.py
evaluate.py
racecar_env.py
train.py
utils.py
visualization.py
wrappers.py
```

2.2 Your Files

You must implement the following files in the **algorithms** directory:

- ppo.py
- sac.py
- td3.py

3 Environment Interface

The RaceCar environment follows the standard Gymnasium API (**reset()**, **step()**, **render()**). In order to examine the details of the environment (i.e., reward function, observation/action vector, etc.), check the Python files in the **dynamics** folder and also **racecar_env.py**, **wrappers.py** files.

To visually inspect the environment, you can run the baseline method using the following command:

```
python evaluate.py --algorithm BASELINE
```

The baseline method uses a PID controller with a Pure Pursuit planner to guide the robot toward the goal.

4 Tasks

4.1 Task 1 - TD3 (Twin Delayed DDPG)

File: `td3.py`

Class: `TD3Agent`

The algorithm learns a deterministic policy and two Q-functions to mitigate the over-estimation bias present in standard DDPG. You will implement the actor and critic networks, the addition of clipped action noise to the target policy, and the delayed updates to the policy and target networks.

4.2 Task 2 - PPO (Proximal Policy Optimization)

File: `ppo.py`

Class: `PPOAgent`

The algorithm learns a stochastic policy using a clipped surrogate objective function. You will implement advantage estimation, the computation of the ratio between the new and old policies, and the clipped loss function to ensure the policy updates remain within a stable trust region.

4.3 Task 3 - SAC (Soft Actor-Critic)

File: `sac.py`

Class: `SACAgent`

The algorithm learns a stochastic policy and two Q-functions while aiming to maximize both expected return and action entropy. You will implement the actor network with the reparameterization trick, the twin critic networks, and the automatic entropy tuning mechanism.

5 Environments

5.1 RaceCar Navigation

The main objective is to navigate the continuous control RaceCar environment using different abstraction levels of control (low-level, high-level, and end-to-end).

- **State Space:** Continuous representation including vehicle dynamics (velocities, heading) and sensory observations from the track.
- **Action Space:** Continuous actions for steering and torque/velocity references depending on the algorithm's control abstraction.
- **Rewards:** Progress along the track, maintaining appropriate velocities, heading angle, and penalties for cross-track errors, collisions etc.
- **Use:** Training and evaluating state-of-the-art continuous reinforcement learning algorithms.

6 Running the Full Simulation

After completing all implementations, you can run the experiments using `train.py` and test the policies with `evaluate.py`.

```
python evaluate.py --algorithm BASELINE
python train.py --algorithm TD3
python evaluate.py models/racecar_td3_final.pt --algorithm TD3
python train.py --algorithm PPO
python evaluate.py models/racecar_ppo_final.pt --algorithm PPO
python train.py --algorithm SAC
python evaluate.py models/racecar_sac_final.pt --algorithm SAC
```

During training, some useful information is printed in the terminal so that you can observe your experiment while the code is still running.

7 Installation

Create and activate a virtual environment, and install the dependencies. All dependencies are shared in the `requirements.txt` file. You can use it to install the required packages and dependencies.

8 Grading Criteria

Task		Points
Task 1	TD3 Implementation	30 pts
Task 2	PPO Implementation	35 pts
Task 3	SAC Implementation	35 pts
Total		100 pts

Tune the hyperparameters in your files to achieve a higher mean reward than the reference baseline implementation. The optimal gain constants and network configurations may be different for each algorithm.

9 Tips and Hints

- First, create a virtual environment and install the required packages.
- Read the docstrings in each skeleton file carefully; all necessary formulas are provided there.
- Before implementing the algorithms, check the environment wrappers that will be used for that specific algorithm.
- After every training, test the learned policy visually by running `evaluate.py` in render mode.

10 Submission

Submit only the following files, their trained & saved models and your report:

- `td3.py`
- `ppo.py`
- `sac.py`
- `models/racecar_ppo_final.pt`
- `models/racecar_td3_final.pt`
- `models/racecar_sac_final.pt`

Do not submit modified versions of the provided files.